



9. Arrays and ArrayLists

Exam Objectives

1. Use a one-dimensional array
2. Create and manipulate an ArrayList
3. Traverse the elements of an ArrayList by using iterators and loops including the enhanced for loop
4. Compare an array and an ArrayList

9.1 Declare, instantiate, initialize and use a one-dimensional array

9.1.1 Declaring array variables

An array is an object that holds a fixed number of values of a given type. You can think of an array as a carton of eggs. If you have a carton with 6 slots, then that carton can hold only six eggs. Each slot of the carton can either have an egg or can be empty. Observe that the carton itself is not an egg. Similarly, if you have an array of size six of `int` values (or `ints`, for short), then that array can hold six `ints` but the array itself is not an `int`.

An array of a given type cannot hold anything else except values of that type. For example, an array of `ints` cannot hold long or double values. Similarly, you can have an array of references of any given type. For example, if you have an array of `Strings`, this array can only hold references to `String` objects. An important point to note here is that even though we call it an "array of strings", it does not actually contain `String` objects. It contains only references to `String` objects. You cannot really have an array that contains actual objects.

Array declaration

When you declare an array variable, you are basically specifying the type of values that you want to deal with through that variable. The way to specify that in Java is to apply `[ ]`, i.e., square brackets to the type of the values. For example, if you want a variable through which you will deal with `int` values, you will write `int[]`. Arrays can be multi-dimensional and there will be one set of opening and closing brackets for each dimension. For example,

```
int i; //i is an int
int[] ia1, ia2; //ia1 and ia2 are one dimensional arrays of ints
int[][] iaa; //iaa is a two dimensional array of ints and so on
```

An array declaration can never include the size of the array. Thus, the following are declarations will not compile:

```
int[2] invalid1;
int[3][] invalid2;
int[][4] invalid3;
```

Unfortunately, the above method is not the only way to declare arrays. Java allows you to apply square brackets to the variable name instead of type name as well. For example,

```
int i, ia[]; //i is an int but ia is a one dimensional array of int values
int[] ia, iaa[]; //ia is a one dimensional array of ints but iaa is a two dimensional
    array of ints and so on
```

Observe that the rule of thumb of one set of square brackets per dimension still holds. In the case of `iaa`, you have one set applied on the type and one set applied on the variable, therefore `iaa` is a two dimensional array.

Arrays of objects are declared the same way. For example,

```
Object[] obja, objaa[]; //obja is a one dimensional array of Objects but objaa is a two
    dimensional array of Objects
String[] strA; //strA is a one dimensional array of Strings
```

Note that the statements shown above only declare array variables. They don't actually create arrays. Array creation and initialization is a topic in its own right and that is what I will discuss next.

9.1.2 Creating and initializing array objects

Creating arrays using array creation expressions

You use the new keyword to create an array object. For example,

```
int[] ia = new int[10]; //an array of ints of size 10
boolean[] ba = new boolean[3]; //an array of booleans of size 3
String[] stra = new String[5]; //an array of Strings of size 5
MyClass[] myca = new MyClass[5]; //an array of MyClass of size 5
```

```
int[] invalid = new int[]; //missing size. will not compile
```

The parts on the right-hand side of = sign in the above statements are called "**array creation expressions**". These expressions merely allocate space to hold **10 ints**, **3 booleans**, **5 string references**, and **5 MyClass references** respectively. Every element of the array is also initialized to its default value automatically by the JVM. The default values of array elements are very straightforward - references are initialized to **null**, numeric primitives to **0**, and booleans to **false**. In the above lines of code, ia is set to point to an array of ten int values and each element of the array is initialized to 0, ba is set to point to an array of three boolean values and each element of the array is initialized to false, stra is set to point to an array of five String references and each element of the array is initialized to null, and finally, myca is set to point to an array of five MyClass references and each element of the array is initialized to null. Observe that all the elements of an array are initialized to the same value. This shows us another important aspect of arrays - that arrays can have **duplicate values**.

It is important to understand that, in the above statements, we are not creating instances of the class of the array elements. In other words, we are not creating instances of String or instances of MyClass. (We are not creating instances of ints or booleans either, for that matter, but since they are primitives, and since primitives are not objects, they don't have instances anyway.)

In Java, arrays, whether of primitives or objects, are objects of specific classes. In other words, an array object is an instance of some class. It not an instance of Object class but since Object is the root of every class in Java, an array object is an Object and all methods of the Object class can be invoked on an array. Let us now look at the following program and its output to know more about the class of the above defined array objects -

```
public class TestClass{
    public static void main(String[] name){
        int[] ia = new int[10];
        boolean[] ba = new boolean[3];
```

```

String[] stra = new String[5];
TestClass[] ta = new TestClass[5];
System.out.println(ia.getClass().getName()+" ,
"+ia.getClass().getSuperclass().getName());
System.out.println(ba.getClass().getName()+" ,
"+ba.getClass().getSuperclass().getName());
System.out.println(stra.getClass().getName()+" ,
"+stra.getClass().getSuperclass().getName());
System.out.println(ta.getClass().getName()+" ,
"+ta.getClass().getSuperclass().getName());
}
}

```

Output:

```

[I , java.lang.Object
[Z , java.lang.Object
[Ljava.lang.String; , java.lang.Object
[LTestClass; , java.lang.Object

```

The output shows that `ia`, which is declared to be of type `int[]`, is not an instance of `int` but of a class named `[I`. `stra`, which is declared to be of type `String[]`, is not an instance of `String` but of a class named `[Ljava.lang.String;` and so on. These names of the classes look weird. Actually, Java cooks up the name of the class of an array by looking at the number of dimensions and the type of the elements. For each dimension, there is one opening square bracket. This is followed by a letter for the class of the elements as per the following table and, if the array is not of a primitive, the name of the class followed by a semi-colon.

Type	Letter
boolean	Z
byte	B
char	C
short	S
int	I
long	J
float	F
double	D
any object	L

Table 9.1.1: Letters used for creating class names of arrays

Based on the above table, the name of the class for `long[][]` (i.e. a two dimensional array of longs) would be `[[J` and the name of the class for `mypackage.SomeClass[][][]` (i.e. a three dimensional array of `mypackage.SomeClass`) would be `[[[Lmypackage.SomeClass;`.

Although the above discussion about the class of arrays is not required for the JFCJA exam, it is good to know anyway.

Creating arrays using array initializers

In the previous section, we created array objects using the `new` keyword. It is possible to create array objects without using the `new` keyword. For example, the arrays that we created above can also be created as follows:

```
int[] ia = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }; //an array of ints of size 10
boolean[] ba = {true, false, false}; //an array of booleans of size 3
String[] str = {"a", "b", "c", "d", "e"}; //an array of Strings of size 5
MyClass[] myca = { new MyClass(), new MyClass(), new MyClass(), new MyClass(), new
    MyClass() } ; //an array of MyClass objects of size 5
```

The parts on the right-hand side of the `=` sign in the above statements are called "**array initializers**". An array initializer is a shortcut that allows you to create as well as initialize each element of the array with the values that you want (instead of the default values that you get when you use array creation expressions).

Since the compiler can find the type of the elements of the array by looking at the declaration (i.e. the left-hand side of the statement), specifying the same on the right-hand side is not required. Similarly, the compiler finds out the length of the array as well by counting the number of values that are specified in the initializer.

Array initializers can also be mixed with array creation expressions. For example:

```
int[] ia = new int[]{ 1, 2, 3, 4, 5 };
```

Observe that size is missing from the expression on the right-hand side. Java figures out the size of the array by counting the number of elements that are specified in the initialization list. In fact, it is prohibited to specify the size if you are specifying individual elements. Therefore, the following is invalid:

```
int[] ia = new int[2]{ 1, 2 }; //will not compile.
```

9.2 Using arrays

9.2.1 Array indexing

In Java, array indexing starts with 0. In other words, the index of the first element of an array is 0. For example, if you have an array variable named `accounts` that refers to an array of 100 `Account` objects, then you can access the first object through `accounts[0]` and the 100th object through `accounts[99]`. `accounts[0]` or `account[99]` are like any other variable of type `Account`.

Similarly, if you have an array variable `ia` pointing an array of 5 ints, the first element can be accessed using `ia[0]` and the last element using `ia[4]`.

`java.lang.ArrayIndexOutOfBoundsException`

If you try to access any array beyond its range, JVM will throw an instance of `ArrayIndexOutOfBoundsException`. For example, if you have `int[] ia = new int[3];` the statements `int i = ia[-1];` and `int i = ia[3];` will cause an `ArrayIndexOutOfBoundsException` to be thrown. Do not worry if you don't know much about exceptions at this point. I will discuss exceptions in detail later.

Arrays of length zero

As strange as it may sound, it is possible to have an array of length 0. For example, `int[] ia = new int[0];` Here, `ia` refers to an array of ints of length 0. There are no elements in this array. It is important to understand that an array of length 0 is not the same as `null`. `ia = null` implies that `ia` points to nothing. `ia = new int[0];` implies that `ia` points to an array of ints whose length is 0.

A good example of an array of length 0 is the `args` parameter of the main method. If you run a class with no argument, `args` will **not** be `null` but will refer to an array of Strings of length 0.

Changing the size of an array

Java arrays are always of fixed size(or length). Once you create an array, you cannot change the number of elements that this array can have. So, for example, if you have created an array of 5 ints and if you have 6 int values to store, you will need to create a new int array. There is no way to increase or decrease the size of an existing array. You may change the array variable to point to a different array, and you can, of course, change the values that an array contains.

9.2.2 Members of an array object

We saw earlier that arrays are actually objects of specific classes. We also saw the names of some of these classes. But what do these classes contain? What are the members of these classes? What functionality do these classes provide? Let us take a look:

Array length

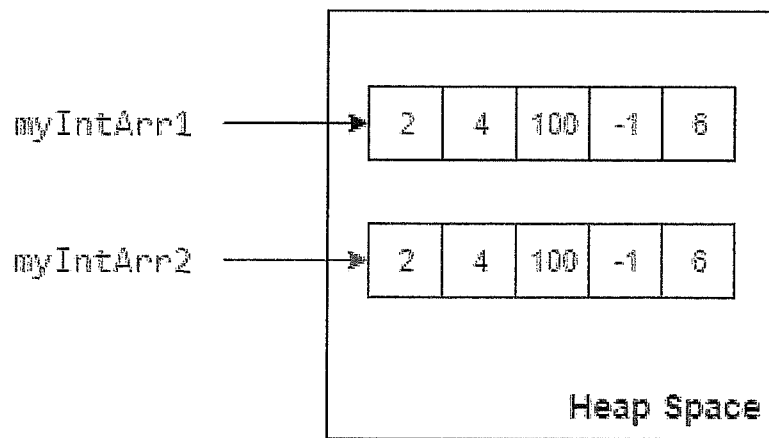
All array classes have one field named **length**, which is of type **int**. This field is **public** and it stores the length of the array. This field is also **final**, which reflects the fact that you cannot change the length of an array after its creation. Since the length of an array can never be less than 0, the value of this field can never be less than 0 either.

Array cloning

All array classes also have a public method named **clone**. This method creates a copy of the array object. Note that it doesn't create copies of the objects referred to by the array elements. It merely creates a new array object of the same length and copies the contents of existing array into the new array. Which means, if the existing array contained primitive values, those values will be copied to the elements of the new array. If the existing array contained references to objects, those references will be copied to the elements of the new array. Thus, the elements of the new array will also point to the same objects. This is also known as "**shallow copy**".

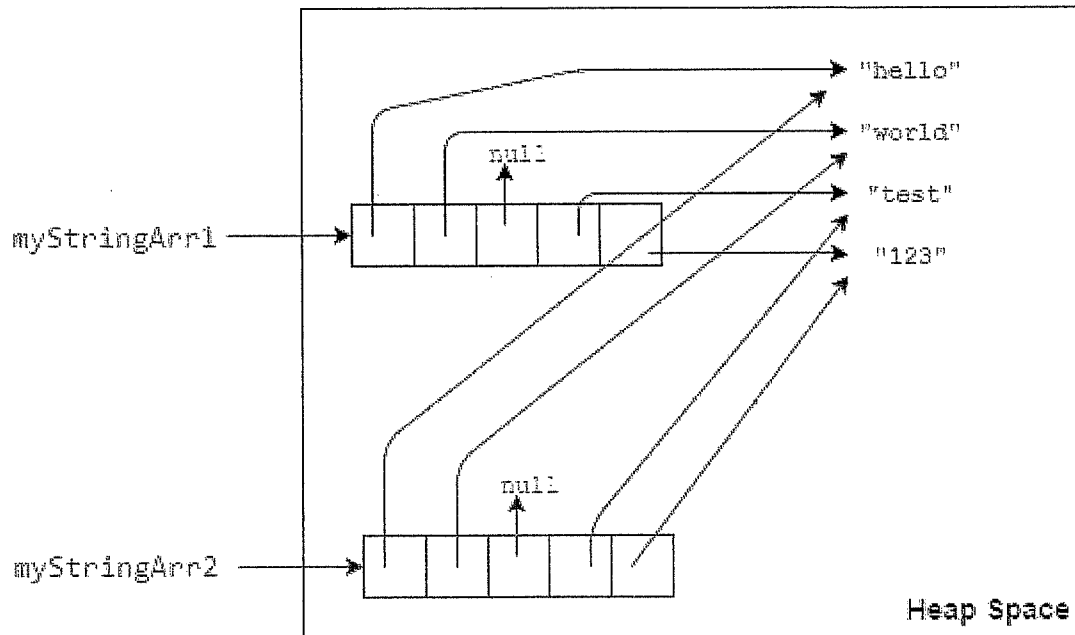
For example, the following figure shows exactly what happens when you clone an array of five ints referred to by the variable `myIntArr1` using the statement `int[] myIntArr2 = (int[])`

`myIntArr1.clone()`; Don't worry about the explicit cast for now, but observe that a new array object with the same number of slots is created and the contents of the slots of the existing array are copied to the slots of the new array.



Cloning an array of ints

The following figure shows what happens when you clone an array of Strings referred to by the variable `myStringArr1` using the statement `String[] myStringArr2 = (String[]) myStringArr1.clone()`; Observe that the references are copied into the new array and that no new String objects are created in the heap space.



Cloning an array of Strings

Shallow copying of an array of ints

Cloning is not required for the exam but it is an important aspect of arrays anyway.

Members inherited from Object class

Remember that since `java.lang.Object` is the root class of all classes, it is the root class of all array classes as well and therefore, array classes inherit all the members of the `Object` class. This includes `toString`, `equals`, and `hashCode` methods.

9.2.3 Runtime behavior of arrays

Although arrays are the simplest of data structures, they are not without their quirks. To use arrays correctly and effectively, you must be aware of the two most important aspects of arrays in Java.

The first is that they are **"reified"**. Meaning, the type checking of arrays and its elements is done at runtime by the JVM instead of at compile time by the compiler. In other words, the type information of an array is preserved in the compiled bytecode for use by the JVM during run time. The JVM knows about the type of an array and enforces type checking on arrays.

For example, if you have an array of `Strings`, the JVM will not let you set any element of that array to point to any object other than a `String`, even though the compiler may not notice such a violation as illustrated by the following code:


```
String[] sa = { "1", "2", "3" };  
Object[] oa = sa;  
oa[0] = new Object();
```

The above code will compile fine. The compiler has no objection when you try to assign an array of Strings to a variable of type array of Objects. It has no objection when you try to set an element of this array of Strings to point to an object that is not a String. But when you run it, the JVM will throw a `java.lang.ArrayStoreException` on the third line. This is because the JVM knows that the array pointed to by `oa` is actually an array of Strings. It will not let you corrupt that array by storing random objects in it.

It is important to understand this concept because it is diametrically opposite to how generics work. This is the reason why arrays and generics don't operate well with each other. You will realize this when you learn about generics later.

The second thing about arrays is that they are "**covariant**". Meaning, you can store a subclass object in an array that is declared to be of the type of its superclass. For example, if you have an array of type `java.lang.Number`, you can store `java.lang.Integer` or `java.lang.Float` objects into that array because both are subclasses of `java.lang.Number`. Thus, the following code will compile and run fine:

```
Number[] na = { 1, 2, 3 };  
na[0] = new Float(1.2f);
```

I haven't seen anyone getting a question on array reification and covariance in the JFCJA exam. However, any discussion about arrays is incomplete without these two points and that is why I have talked about them here.

9.2.4 Uses of arrays

Arrays are quite powerful as a **data structure** but they are somewhat primitive as a **data type**. As we saw earlier, arrays have only one field and merely a couple of methods. But because of their simplicity, arrays are used as building blocks for other data types and data structures. For example, the `String` data type is built upon an array of chars. So are `StringBuffer` and `StringBuilder`. These higher level classes really only wrap an array of characters and provide methods for manipulating that array.

Arrays are also used extensively for building higher level data structures such as `List`, `Stack`, and `Queue`. You may have come across the `ArrayList` class (this class is also on the exam, by the way, and I will discuss it later). Guess what, it is a `List` that manages its collection of objects using an array inside. Since these classes provide a lot of additional features on top of arrays, more often than not, it is these classes that get used in application programs rather than raw arrays.

java.util.Arrays class

Java standard library does include a utility class named **Arrays** in package **java.util** that makes working with raw arrays a little easier. `java.util.Arrays` class contains a large number of static utility methods for manipulating any given array object. Although you are not required to know about this class for the JFCJA exam, it is a good idea to browse through the JavaDoc API description of `Arrays` class to see what you can do with arrays.

9.3 Create and manipulate an ArrayList

9.3.1 Introduction to Collections and Generics

Processing multiple objects of the same kind in the same way is often a requirement in applications. Printing the names of all the Employees in a list of Employees, computing interest for a list of Accounts, or something as simple as computing the average of a list of numbers, require you do deal with a list of objects instead of one single object. You have already seen a data structure that is capable of holding multiple objects of the same kind - array. An array lets you hold references to multiple objects of a type and pass them around as a bunch. However, arrays have a couple of limitations. First, an array cannot grow or shrink in length after it is created. If you create an array of 10 Employees and later find out you have 11 Employees to hold, you will need to create a new array with 11 slots. You can't just add one more slot in an existing array. Second, inserting a value in the middle of an array is a bit difficult. If you want to insert an element just before the last element, you will have to first shift the last element to the next position and then put the value. Imagine if you want to insert an element in the middle of the list!

Although both of the limitations can be overcome by writing some extra code, these requirements are so common that writing the same code over and over again is just not a good idea. `java.util.ArrayList` is a class that incorporates these, and several other, features out of the box. The following is an example that shows how easy it is to manage a list of objects using an `ArrayList`:

```
import java.util.ArrayList;
public class TestClass{
    public static void main(String[] args){
        ArrayList al = new ArrayList();

        al.add("alice"); //[alice]
        al.add("bob");  //[alice, bob]
        al.add("charlie"); //[alice, bob, charlie]

        al.add(2, "david"); //[alice, bob, david, charlie]

        al.remove(0); //[bob, david, charlie]

        for(Object o : al){ //process objects in the list
            String name = (String) o;
            System.out.println(name+" "+name.length());
        }
    }
}
```

```
//dump contents of the list
System.out.println("All names: "+al);
}
}
```

You will get a few warning messages when you compile the above program. Ignore them for now. It prints the following output when run:

```
bob 3
david 5
charlie 7
All names: [bob, david, charlie]
```

The above program illustrates the basics of using an `ArrayList`, i.e., adding objects, removing, iterating over them, and printing the contents of the `ArrayList`. I suggest you write a program that does the same thing using an array of strings instead of an `ArrayList`. This will give you an appreciation for the value that `ArrayList` adds over a raw array.

I showed you the above code just to give you a glimpse of one of the readymade classes in the Java library. There is a lot more that you can do with it but before we get to that, you need to understand the bigger picture, the grand scheme of things, into which classes such as `ArrayList` fit.

Collections and Collections API

`ArrayList` belongs to a category of classes called "**collections**". A "**collection**" is nothing but a group of objects held up together in some form. You can visualize a collection as a bag in which you put several things. Just like you use a bag to take various grocery items from a store to your home, you put the objects in a collection so that you can pass them around to other objects and methods. A bag knows nothing about the items themselves. It merely holds the items. You can add items to it and take items out of it. It doesn't care about the order in which you put the items or whether there are any duplicates or how they are stored inside the bag. The behavior of a bag is captured by an interface called `Collection`, which exists in the `java.util` package.

Now, what if you want a collection that ensures objects can be retrieved from it in the same order in which they were added to it? Or what if you want a collection that does not allow duplicate objects? You can think of several features that can be added on top of a basic collection. As you have already learnt previously, subclassing/subinterfacing is the way you extend the behavior of an existing class or interface. The Java standard library takes the same route here and extends `java.util.Collection` interface to provide several interfaces with various features. In fact, the Java standard library provides a huge tree of interfaces along with classes that implement those interfaces. These classes and interfaces are collectively known as the "**Collections API**".

So what has `ArrayList` got to do with this, you ask? Well, `ArrayList` is a class that implements one of the specialized collection interfaces called `List`. `java.util.List` **extends** `java.util.Collection` to add the behavior of ordering on top of a simple collection. A `List`

is supposed to remember the order in which objects are added to the collection. Okay, so that takes care of the "List" part of ArrayList, what about the "Array" part? Again, as you have learnt in previous chapters, an interface merely describes a behavior. It doesn't really implement that behavior. You need a class to implement the behavior. In this case, ArrayList is the class that implements the behavior described by `java.util.List` and to implement this behavior, it uses an array. Hence the name **ArrayList**. Remember I talked about writing code to overcome the limitation of an array that it is not flexible enough to increase in size automatically? ArrayList contains that code. When you add an object to an ArrayList, it checks whether there is space left in the array. If not, it allocates a new array with a bigger length, copies the elements from the old array to this new array, and then adds the passed object to the new array. Similarly, it contains code for inserting an object in the middle of the array. All this is transparent to the user of an ArrayList. A user simply invokes methods such as `add` and `remove` on an ArrayList without having any knowledge of the array that is being used inside to store the object.

Generics

Now, regarding the warning messages that I asked you to ignore. Observe the for loop in the above code. The type of the loop variable `o` is `Object` (and not `String`). To invoke the `length` method of `String` on `o`, we need to cast `o` to `String`. This is because, in the same way that a bag is unaware of what is inside of it, so too does the ArrayList object have no knowledge about the type of the objects that have been added to it. It is the responsibility of the programmer to cast the objects that they retrieve from the list to appropriate types. In this program, we know that we have added Strings to this list and so we know that we can cast the object that we retrieve from this list to `String`. But what if we were simply given a pre-populated list as an argument? Since the list would have been populated by another class written by someone else, what guarantee would we have about the type of objects we find in the list? None, actually. And as you are aware, if we try to cast an object to an inappropriate type, we get a `ClassCastException` at run time. Getting a `ClassCastException` at run time would be a coding mistake that will be discovered only at run time. Discovering coding mistakes at run time is not a good thing and the compiler is only trying to warn us about this potential issue by generating warning messages while compiling our code.

The first warning message that it prints out is, "Warning: unchecked call to `add(E)` as a member of the raw type `java.util.ArrayList`" for the code `al.add("alice");` at line 6. It prints the same warning every time an object is added to the list. The compiler is trying to tell us that it does not know what kind of objects this ArrayList is supposed to hold and that it has no way of checking what we are adding to this list. By printing out the warning, the compiler is telling us that it will not be held responsible for the objects that are being put in this list. Whoever wants to retrieve objects from this list must already know what type of objects are present inside the list and must cast those objects appropriately upon retrieval at their own risk. In other words, this list is basically **"type unsafe"**. It is unsafe because it depends on assumptions made by humans about the contents of the list. This assumption is not checked or validated by the compiler for its truthfulness. One can put any kind of object in the list and others will not realize until they are hit with a `ClassCastException` at run time when they try to cast the retrieved object to its expected type.

Java solves this problem by allowing us to specify the type of objects that we are going to store in a list while declaring the type of the list. If, instead of writing `ArrayList al = new ArrayList();`, we write `ArrayList<String> al = new ArrayList<String> ();` all the warning messages go away. The compiler now knows that the `ArrayList` pointed to by `al` is supposed to hold only `Strings`. Therefore, it will be able to keep a watch on the type of objects the code tries to add to this list. If it sees any code that attempts to add anything that is not a `String` to the list, it will refuse to compile the code. The error message generated by the following code illustrates this point:

```
ArrayList<String> al = new ArrayList<String>();  
al.add(new Object());
```

The error message is:

```
Error: no suitable method found for add(java.lang.Object)  
    method java.util.Collection.add(java.lang.String) is not applicable
```

The compiler will not let you corrupt the list by adding objects that this list is not supposed to keep. Similarly, the compiler will not let you make incorrect assignments either. Here is an example:

```
List<String> al = new ArrayList<String>(); //al points to a List of Strings  
al.add("hello"); //valid  
String s = al.get(0); //valid, no cast required  
Object obj = al.get(0); //valid because a String is-a Object  
Integer in = al.get(0); //Invalid. Will not compile
```

The error message is:

```
Error: incompatible types: java.lang.String cannot be converted to java.lang.Integer
```

Observe that we supplied information about the type of object the list is supposed to keep by appending `<String>` to the variable declaration and the `ArrayList` creation. This way of specifying the type information is a part of a feature introduced in Java 5 known as "**generics**".

The above discussion should tell you that the topics of `List/ArrayList`, `Collections`, and `Generics` are interrelated. All three of them combine together to provide you a powerful mechanism to write type safe and bug free code. However, the JFCJA exam requires you to know only the basic usage of the `ArrayList` class. You are not expected to know about `Collections` and `Generics` and so, I will not talk about them anymore in this book.

9.3.2 ArrayList API

As discussed earlier, `ArrayList` is one of the implementation classes of the `List` interface. It has three constructors:

1. `ArrayList()`: Constructs an empty list with an initial capacity of 10. Just like you saw with the `StringBuilder` class, capacity is simply the size of the initial array that is used to store

the objects. As soon as you add the 11th object, a new array with bigger capacity will be allocated and all the existing objects will be transferred to the new array.

2. `ArrayList(Collection c)`: Constructs a list containing the elements of the specified collection.
3. `ArrayList(int initialCapacity)`: Constructs an empty list with the specified initial capacity. This constructor is helpful when you know the approximate number of objects that you want to add to the list. Specifying an initial capacity that is greater than the number of objects that the list will hold improves performance by avoiding the need to allocate a new array every time it uses up its existing capacity. It is possible to increase the capacity of an `ArrayList` even after it has been created by invoking `ensureCapacity(int n)` on that `ArrayList` instance. Calling this method with an appropriate number before inserting a large number of elements in the `ArrayList` improves performance of the add operation by reducing the need for incremental reallocation of the internal array. The opposite of `ensureCapacity` is the `trimToSize()` method, which gets rid of all the unused space by reducing its capacity to the match the number of elements in the list.

Here are a few declarations that you may see on the exam:

```
1. List list = new ArrayList(); //ok because ArrayList is a List
2. ArrayList<String> al2 = new ArrayList<String>(list); //copying an existing list,
   observe the diamond operator
3. List<Student> list1 = new ArrayList<>(); //ok, list1 is of type List<Student>
```

The third declaration is interesting. We have used `<Student>` on the left side of `==` but only `<>` on the right side. `<>` is called the "diamond operator". It just means that the compiler should take type information from the left side of `==`. Since we have already declared `list1` to be a `List` of `Student` objects, there is no need for us to type this information again on the right side. Thus, all it does is save us from typing a few extra keystrokes.

Important methods of ArrayList

`ArrayList` has quite a lot of methods. However, since `ArrayList` implements `List` (which, in turn, extends `Collection`), several of `ArrayList`'s methods are declared in `List` and `Collection` interfaces. The exam does not expect you to make a distinction between the methods inherited from `List`/`Collection` and the methods declared in `ArrayList`.

The following are the ones that you need to know for the exam:

1. `String toString()`: Well, `toString` is not really the most important method of `ArrayList` but since we will be depending on its output in our examples, it is better to know about it anyway. `ArrayList`'s `toString` first gets a string representation for each of its elements (by invoking `toString` on them) and then combines into a single string that starts with `[` and ends with `]`. For example, the following code prints `[a, b, c]`:

```
var al = new ArrayList<String>();  
al.add("a");  
al.add("b");  
al.add("c");  
System.out.println(al);
```

Observe the order of the elements in the output. It is the same as the order in which they were added in the list. Calling `toString` on an empty `ArrayList` gets you `[]`. I will use the same format to show the contents of a list in code samples.

Methods that add elements to an `ArrayList`:

1. `boolean add(E e)`: Appends the specified element to the end of this list. As discussed in the Generics section, `E` is just a place holder for whichever type you specify while creating the `ArrayList`. For example, if you create an `ArrayList` of Strings, i.e., `ArrayList<String>`, `E` stands for `String`.

This method is actually declared in `Collection` interface and the return value is used to convey whether the collection was changed as a result of calling this method. In case of an `ArrayList`, the `add` method always adds the given element to the list (even if the element is null), which means it changes the collection every time it is invoked. Therefore, it always returns `true`.

2. `void add(int index, E element)`: Inserts the specified element at the specified position in this list. The indexing starts from 0 and the maximum value of `index` can be `size`. Therefore, if you call `add(0, "hello")` on an list of Strings, "hello" will be inserted at the first position.
3. `boolean addAll(Collection<? extends E> c)`: Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's `Iterator`. Again, for the purpose of the Part 1 exam, you don't need to worry about the "`? extends E`" or the `Iterator` part. You just need to know that you can add all the elements of one list to another list using this method. For example,

```
ArrayList<String> sList1 = new ArrayList<>(); //observe the usage of the diamond  
    operator  
sList1.add("a"); //[a]  
ArrayList<String> sList2 = new ArrayList<>();  
sList2.add("b"); //[b]  
sList2.addAll(sList1); //sList2 now contains [b, a]
```

4. `boolean addAll(int index, Collection<? extends E> c)`: This method is similar to the one above except that it inserts the elements of the passed list in the specified collection into this list, starting at the specified position. For example,

```
ArrayList<String> sList1 = new ArrayList<>();  
sList1.add("a"); //[a]
```

```
ArrayList<String> sList3 = new ArrayList<>();  
sList3.add("b"); //[b]  
sList3.addAll(0, sList1); //sList3 now contains [a, b]
```

Methods that remove elements from an ArrayList:

1. `E remove(int index)`: Removes the element at the specified position in this list. For example,

```
ArrayList<String> list = ... // an ArrayList containing [a, b, c]  
String s = list.remove(1); //list now has [a, c]
```

It returns the element that has been removed from the list. Therefore, `s` will be assigned the element that was removed, i.e., "b". If you pass an invalid `int` value as an argument (such as a negative value or a value that is beyond the range of the list, i.e., `size-1`), an `IndexOutOfBoundsException` will be thrown.

2. `boolean remove(Object o)`: Removes the first occurrence of the specified element from this list, if it is present. For example,

```
ArrayList<String> list = ... // an ArrayList containing [a, b, a]  
list.remove("a"); //[b, a]
```

Observe that only the first `a` is removed.

You have to pay attention while using this method on an `ArrayList` of `Integers`. Can you guess what the following code will print?

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList( new Integer[]{1, 2, 3 }  
    ));  
list.remove(1);  
System.out.println(list);  
list = new ArrayList<>(Arrays.asList( new Integer[]{1, 2, 3 } ));  
list.remove(new Integer(1));  
System.out.println(list);
```

The output is:

```
[1, 3]  
[2, 3]
```

Recall the rules of method selection in the case of overloaded methods. When you call `remove(1)`, the argument is an `int` and since a `remove` method with `int` parameter is available, this method will be preferred over the other `remove` method with `Object` parameter because invoking the other method requires boxing `1` into an `Integer`.

This method returns `true` if an element was actually removed from the list as a result of this call. In other words, if there is no element in the list that matches the argument, the method will return `false`.

3. `boolean removeAll(Collection<?> c)`: Removes from this list all of its elements that are contained in the specified collection. For example the following code prints [c]:

```
ArrayList<String> al1 = new ArrayList<>(Arrays.asList( new String[]{"a", "b",  
    "c", "a" } ));  
ArrayList<String> al2 = new ArrayList<>(Arrays.asList( new String[]{"a", "b" } ));  
al1.removeAll(al2);  
System.out.println(al1); //prints [ c ]
```

Observe that unlike the `remove(Object obj)` method, which removes only the first occurrence of an element, `removeAll` removes all occurrences of an element. This method returns `true` if an element was actually removed from the list as a result of this call.

4. `void clear()`: Removes all of the elements from this list.

Methods that replace an element in an ArrayList:

1. `E set(int index, E element)`: Replaces the element at the specified position in this list with the specified element. It returns the element that was replaced. Example:

```
ArrayList<String> al = ... // create a list containing [a, b, c]  
String oldVal = al.set(1, "x");  
System.out.println(al); //prints [a, x, c]  
System.out.println(oldVal); //prints b
```

Methods that read an ArrayList without modifying it:

1. `boolean contains(Object o)`: The object passed in the argument is compared with each element in the list using the `equals` method. A `true` is returned as soon as a match is found, a `false` is returned otherwise. Here are a couple of examples:

```
ArrayList<String> al = new ArrayList<>();  
al.addAll(Arrays.asList( new String[]{"a", null, "b", "c" } ));  
System.out.println(al.contains("c")); //prints true  
System.out.println(al.contains("z")); //prints false  
System.out.println(al.contains(null)); //prints true
```

Observe that it does not throw a `NullPointerException` even if you pass it a `null`. In fact, a `null` argument matches a `null` element.

2. `E get(int index)`: Returns the element at the specified position in this list. It throws an `IndexOutOfBoundsException` if an invalid value (i.e. a value less than 0 or greater than `size-1`) is passed as an argument.
3. `int indexOf(Object o)`: The object passed in the argument is compared with each element in the list using the `equals` method. The index of the first element that matches is returned. If this list does not contain a matching element, `-1` is returned. Here are a couple of examples:

```
ArrayList<String> al = new ArrayList<>();
al.addAll(Arrays.asList( new String[]{"a", null, "b", "c", null } ));
System.out.println(al.indexOf("c")); //prints 3
System.out.println(al.indexOf("z")); //prints -1
System.out.println(al.indexOf(null)); //prints 1
```

Observe that just like `contains`, `indexOf` does not throw a `NullPointerException` either even if you pass it a `null`. A `null` argument matches a `null` element.

4. `boolean isEmpty()`: Returns `true` if this list contains no elements.
5. `int size()`: Returns the number of elements in this list. Recall that to get the number of elements in a simple array, you use the variable named `length` of that array.

The examples that I have given above are meant to illustrate only a single method. In the exam, however, you will see code that uses multiple methods. Here are a few points that you should remember for the exam:

1. **Adding nulls**: `ArrayList` supports `null` elements.
2. **Adding duplicates**: `ArrayList` supports duplicate elements.
3. **Exceptions**: None of the `ArrayList` methods except `toArray(T[] a)` throw `NullPointerException`. They throw `IndexOutOfBoundsException` if you try to access an element beyond the range of the list.
4. **Method chaining**: Unlike `StringBuilder`, none of the `ArrayList` methods return a reference to the same `ArrayList` object. Therefore, it is not possible to chain method calls.

Here are a few examples of the kind of code you will see in the exam. Try to determine the output of the following code snippets when they are compiled and executed:

1. --

```
var al = new ArrayList<Integer>(); //observe that the type specification is on
    the right side
al.add(1).add(2);
System.out.println(al);
```

2. --

```
ArrayList<String> al = new ArrayList<>(); //observe the usage of the diamond
    operator
if( al.add("a") ){
    if( al.contains("a") ){
        al.add(al.indexOf("a"), "b");
    }
}
System.out.println(al);
```

3. --

```
ArrayList<String> al = new ArrayList<>();  
al.add("a"); al.add("b");  
al.add(al.size(), "x");  
System.out.println(al);
```

4. --

```
var list1 = new ArrayList<String>();  
var list2 = new ArrayList<String>();  
list1.add( "a"); list1.add("b");  
list2.add("b"); list2.add("c"); list2.add("d");  
list1.addAll(list2);  
list1.remove("b");  
System.out.println(list1);
```

5. --

```
ArrayList<String> list1 = new ArrayList<>();  
ArrayList<String> list2 = new ArrayList<>();  
list1.add( "a"); list1.add("b");  
list2.add("b"); list2.add("c"); list2.add("d");  
list1.addAll(list2);  
System.out.println(list1);  
list1.remove("b");  
System.out.println(list1);
```

6. --

```
ArrayList<String> list1 = new ArrayList<>();  
ArrayList<String> list2 = new ArrayList<>();  
list1.add( "a"); list1.add("b");  
list2.add("b"); list2.add("c"); list2.add("d");  
list1.addAll(list2);  
list1.removeAll("b");  
System.out.println(list1);
```

4. Now, `it.hasNext()` returns `false` because there are no more elements waiting to be iterated upon in the list. Thus, the loop ends.

9.5 ArrayList vs array

You may get a few theoretical questions in the exam about the advantages and disadvantages of an ArrayList over an array. You have already seen all that we can do with ArrayLists and arrays, so, I am just going to summarize their advantages and disadvantages here.

Advantages of ArrayList

1. **Dynamic sizing** - An ArrayList can grow in size as required. The programmer doesn't have to worry about the length of the ArrayList while adding elements to it.
2. **Type safety** - An ArrayList can be made type safe using generics.
3. **Readymade features** - ArrayList provides methods for searching and for inserting elements anywhere in the list.

Disadvantages of ArrayList

1. **Higher memory usage** - An ArrayList generally requires more space than is necessary to hold the same number of elements in an array.
2. **No type safety** - Without generics, an ArrayList is not type safe at all.
3. **No support for primitive values** - ArrayLists cannot store primitive values while arrays can. This disadvantage has been mitigated somewhat with the introduction of autoboxing in Java 5, which makes it possible to pass primitive values to various methods of an ArrayList. However, autoboxing does impact performance.

Similarities between ArrayLists and arrays

1. **Ordering** - Both maintain the order of their elements (meaning, an element stored at a given position aka index can be accessed by specifying that position) and allow retrieval of elements based on an index, which ranges from 0 (inclusive) to length (exclusive). Since elements in an array and an ArrayList can be accessed through an index, both can be traversed in either direction.
2. **Duplicates** - Both allow duplicate elements to be stored.
3. **nulls** - Both allows nulls to be stored.
4. **Performance** - Since an ArrayList is backed by an array internally, there is no difference in performance of various operations such as searching on an ArrayList and on an array.
5. **Thread safety** - Neither of them are thread safe. Don't worry, thread safety is not on the exam, but you should be aware of the fact that accessing either of them from multiple threads, may produce incorrect results in certain situations.

9.6 Exercise

1. Create an array of booleans of length 3 inside the main method. Print the elements of the array without initializing the array elements explicitly. Observe the output.
2. Given `int[] first = new int[3];`, `int[] second = {};`, and `int[] third = null;`, print out the length of the three arrays and print out every element of the three arrays.
3. Create an array of chars containing four values. Write assignment statements involving the array such that the first element of the array will contain the value of the second element, second element will have the value that was there in the third element. and third element will contain the value of the fourth element.
4. Declare and initialize an array of length 4 of type array of Strings without using the new keyword such that no two arrays of Strings have the same length. Print the length of all of the arrays one by one (including the length of the two dimensional array).
5. Define a simple class named Data with a public instance field named value of type int. Create and initialize a Data variable named d in TestClass's main. Create an array of Data of length 3 and initialize each of its elements with the same Data instance. Use any of the array elements to update the value field of the Data object. Print out the value field of the Data object using the three elements of the array. Finally, print the value field of the original Data using the variable d.
6. Create an array of ints and an ArrayList of Integers. Add alternate elements of the array to the list. Remove all even values from the list.
7. Create an ArrayList with some values. Print the first and the last element of the list. Insert a new value in the middle of the list.
8. Create an ArrayList and add all the arguments given on the command line to that list. Search for a particular argument in the list and print its index. Check if a particular argument is present in the list and if present, remove it from the list.

